

Composite Design Pattern

Category: Structural | **GoF Reference:** Gang of Four — Design Patterns (1994), Chapter 4

Table of Contents

1. [Intent](#)
 2. [The Problem It Solves](#)
 3. [When to Use](#)
 4. [Structure — ASCII UML](#)
 5. [Package Structure](#)
 6. [The Players](#)
 7. [Code Walkthrough — Every File, Every Line](#)
 8. [IUnit.java \(Component\)](#)
 9. [Soldier.java \(Leaf\)](#)
 10. [Squad.java \(Composite\)](#)
 11. [BattleFormationDesignPattern.java \(Driver / Client\)](#)
 12. [Why These Design Decisions](#)
 13. [Execution Flow Trace](#)
 14. [Expected Output](#)
 15. [How to Run](#)
 16. [Real-World Use Cases](#)
 17. [Extending This Example — Deeper Nesting](#)
 18. [Summary](#)
-

Intent

The **Composite Design Pattern** lets you compose objects into tree structures to represent part-whole hierarchies, and then lets clients treat individual objects and compositions of objects **uniformly through a common interface**.

In plain terms: you define a single interface (the *Component*). Both a simple, indivisible object (the *Leaf*) and a complex container of objects (the *Composite*) implement that same interface. Client code never needs to know whether it is talking to a leaf or a composite — it always calls the same method on the same interface type, and the right behaviour happens automatically through polymorphism and recursion.

This eliminates branching logic in the caller. The caller does not ask "is this a single soldier or a squad?" — it just calls `muster()` and the object figures out what to do.

The Problem It Solves

Consider a battlefield command structure. You have individual soldiers and squads. A squad can contain individual soldiers, and it can also contain other squads (a platoon of squads, for example). The client wants to muster (roll-call and report) everything under a given top-level formation.

Without the Composite pattern, client code must distinguish between a soldier and a squad manually:

```
// Without Composite – ugly, brittle, hard to extend
void reportUnit(Object unit) {
    if (unit instanceof Soldier) {
        Soldier s = (Soldier) unit;
        System.out.println("Soldier: " + s.getCallsign());
    } else if (unit instanceof Squad) {
        Squad sq = (Squad) unit;
        System.out.println("Squad: " + sq.getName());
        for (Object member : sq.getMembers()) {
            reportUnit(member); // manual recursion scattered in client code
        }
    }
}
// What if we add a Vehicle unit? Every caller must be updated.
}
```

Problems with this approach:

- Every place in the codebase that handles battlefield units needs its own `instanceof` chain.
- Adding a new unit type (vehicle, artillery battery, drone wing) requires hunting down every such chain and patching it.

- The recursion logic — walking into sub-squads — lives in the caller, not in the data structure. This makes callers complex and duplicates recursion logic across the codebase.
- Unit testing is harder because you cannot test a `Squad` node in isolation from the traversal logic.

With the Composite pattern, client code reduces to one line:

```
// With Composite – clean, uniform, extensible
formation.muster();
```

The `muster()` call works whether `formation` is a single `Soldier` or a deeply nested `Squad` tree. New unit types just implement `IUnit` — no caller changes required.

When to Use

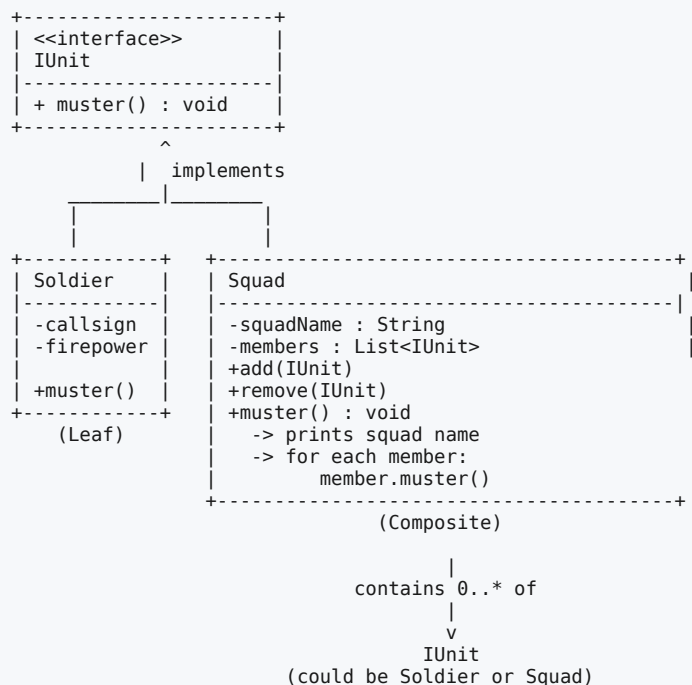
Use the Composite pattern when:

1. **You have a tree (part-whole) hierarchy.** The domain naturally decomposes into nodes that are either leaves (no members) or branches (containing other nodes of the same type).
2. **You want clients to ignore the difference between individual objects and compositions.** The client should call the same operation regardless of whether it is dealing with a single unit or a formation of units.
3. **Adding new node types should not require changes to existing client code.** Open/Closed Principle: open for extension (add a new leaf or composite class), closed for modification (callers do not change).
4. **Recursive traversal should be encapsulated in the data structure, not in the caller.** Each composite node is responsible for traversing its own members.
5. **The depth of nesting is not known at compile time.** The formation can be arbitrarily deep and can change at runtime (soldiers reassigned, squads regrouped).

Do not use Composite when:

- Every node in the hierarchy is a leaf (no containment). A plain list or array is simpler.
- You need to enforce strict constraints on which types of members a composite can hold (e.g., a `SpecialOpsSquad` can only contain `SniperUnit`, not any arbitrary `IUnit`). The pattern's uniformity works against such type-level restrictions.

Structure — ASCII UML



The critical observation in the UML is that `Squad` holds a `List<IUnit>` — a list of the *interface*, not the concrete `Soldier` class. This means a `Squad` can hold any mix of `Soldier` objects and other `Squad` objects (because `Squad` itself

implements `IUnit`). This is what enables the recursive, arbitrarily deep formation structure.

Package Structure

```
com.design.patterns.composite
├── component
│   └── IUnit.java           (Component – the shared contract)
├── leaf
│   └── Soldier.java        (Leaf – indivisible unit)
├── composite
│   └── Squad.java          (Composite – branch unit)
└── BattleFormationDesignPattern.java (Driver – client / entry point)
```

The three sub-packages exist for clarity and separation of concerns:

- **component** — Holds only the interface. This is the contract that every participant in the pattern must fulfill. It has no dependencies on `leaf` or `composite` . Other packages depend on it; it depends on nothing in the pattern.
- **leaf** — Holds concrete leaf implementations. Depends only on `component` . A leaf is by definition the terminating node — it has no members and does not import `composite` .
- **composite** — Holds the branch implementation. Depends on `component` (to hold `List<IUnit>`) but does not import `leaf` directly. It works with the interface, which means it can hold any future leaf type without modification.
- **Root package** — Holds only the driver / demo class. This is the only place where concrete types (`Soldier` , `Squad`) are instantiated. It is the "composition root" — the one place in the application that knows about all the concrete implementations.

This layering enforces the Dependency Inversion Principle: high-level policy (`Squad` 's traversal logic) depends on the abstraction (`IUnit`), not on the concrete `Soldier` class.

The Players

component/IUnit	the Component – declares the shared <code>muster()</code> contract
leaf/Soldier	the Leaf – a terminal unit holding a callsign and a firepower rating
composite/Squad	the Composite – a branch unit holding 0..* <code>IUnit</code> members
BattleFormationDesignPattern	the Driver / Client – builds the formation and triggers <code>muster()</code>

Code Walkthrough — Every File, Every Line

`IUnit.java` — Component Interface

```
package com.design.patterns.composite.component;
```

Declares the Java package. `component` is a sub-package of the pattern's root. In a Maven project, this corresponds to the directory `src/main/java/com/design/patterns/composite/component/` .

```
public interface IUnit {
```

Declares a `public` interface named `IUnit` . It is `public` so that classes in other packages (`leaf` , `composite` , and the root driver) can implement or reference it. Using an `interface` rather than an abstract class is a deliberate choice: it keeps the contract as pure behaviour with no shared state, and it lets `Soldier` or `Squad` extend another class later if ever needed, since Java supports only single class inheritance but multiple interface implementation.

```
    void muster();
```

The single method in the contract. Both participants — `Soldier` (leaf) and `Squad` (composite) — must provide a concrete `muster()` implementation. It returns `void` because its purpose here is a side effect (printing to the console). The fact that this is the *only* method in the interface is intentional: the Composite pattern works best when the shared interface is minimal. A broader interface (e.g., adding `getRank()` , `getOrders()`) would force leaf units to implement methods that may not make sense for them.

```
}
```

Closes the interface declaration.

Soldier.java — Leaf

```
package com.design.patterns.composite.leaf;
```

Places `Soldier` in the `leaf` sub-package. Leaf classes are the indivisible, terminal nodes of the formation. They have no members and do not hold a reference to any `IUnit`.

```
import com.design.patterns.composite.component.IUnit;
```

Imports the `IUnit` interface from the `component` package. This is the *only* import in this file. `Soldier` does not know about `Squad`, and it does not need to. The leaf's dependency graph is minimal: it knows only about the contract it fulfills.

```
public class Soldier implements IUnit {
```

Declares `Soldier` as a concrete public class that fulfills the `IUnit` contract. The `implements IUnit` clause is the key statement — it enrolls this class into the Composite pattern. From this point, any variable of type `IUnit` can hold a `Soldier` instance.

```
    private String callsign;
```

The soldier's callsign (e.g., "Rook"). It is `private` — correct encapsulation. No other class directly accesses the raw field. `muster()` is the only public operation, so no getter is needed.

```
    private int firepower;
```

The soldier's firepower rating, stored as a plain `int`. `private` for the same encapsulation reasons. `int` is sufficient for the ratings used in this demo.

```
    public Soldier(String callsign, int firepower) {
```

A public two-argument constructor. It is `public` so the driver (in the root package) can instantiate it. Both `callsign` and `firepower` are required — there are no setters, so a `Soldier`'s identity is fixed after construction.

```
        this.callsign = callsign;
```

Assigns the constructor parameter `callsign` to the instance field `this.callsign`. The `this.` qualifier is necessary because the constructor parameter and the field share the same identifier; without it, `callsign = callsign` would be a no-op that writes the parameter back to itself.

```
        this.firepower = firepower;
```

Same pattern — assigns the constructor parameter `firepower` to the instance field `this.firepower`.

```
    }
```

Closes the constructor body.

```
    @Override
```

The `@Override` annotation tells the compiler that `muster()` is intended to override a method declared in a supertype — here, the `IUnit` interface. If the signature does not match (e.g., due to a typo), the compiler emits an error instead of silently creating an unrelated method. Always use `@Override` when implementing interface methods — it is a safety net.

```
    public void muster() {
```

Implements the `muster()` method required by the `IUnit` contract. It is `public` because interface methods are implicitly public, and an implementing method must be at least as visible as the interface method.

```
System.out.println("Soldier : " + callsign + " with firepower : " + firepower);
```

Prints the soldier's callsign and firepower rating to standard output, e.g. `Soldier : Rook with firepower : 120`. `callsign` and `firepower` refer to the instance fields.

```
    }  
}
```

Closes `muster()` and the `Soldier` class.

Squad.java — Composite

```
package com.design.patterns.composite.composite;
```

Places `Squad` in the `composite` sub-package — the branch/container role of the pattern.

```
import java.util.ArrayList;  
import java.util.List;
```

Imports the standard Java collection types. `List` is the interface used for the field declaration (programming to interfaces), and `ArrayList` is the concrete implementation used to back it.

```
import com.design.patterns.composite.component.IUnit;
```

Imports the shared component interface. `Squad` holds a `List<IUnit>` and iterates over it — the only pattern-related type it depends on. Crucially, `Squad` does not import `Soldier`. It works purely with the interface, so it can hold any present or future `IUnit` implementation without modification.

```
public class Squad implements IUnit {
```

Declares `Squad` as a concrete public class that also implements `IUnit`. This is the key insight of the Composite pattern: the composite (container) implements the same interface as the leaf. This is what allows a `Squad` to be held inside another `Squad`'s member list — because `Squad` is itself an `IUnit`.

```
    private final String squadName;
```

The squad's display name (e.g., "Alpha Command", "Recon Team"). It is `private` for encapsulation and `final` because a squad's name never changes after construction — there is no rename operation in this model. Unlike `Soldier`, there is no `firepower` field: squads in this model are purely structural containers, not measured by an intrinsic firepower value of their own.

```
    private final List<IUnit> members = new ArrayList<>();
```

This is the *defining field* of the Composite pattern. It is `private` (properly encapsulated — no other class can reach in and mutate it directly) and `final` (the list reference itself never changes; only its contents do, through `add` / `remove`). It is typed as a list of the interface `IUnit`, not of the concrete `Soldier` type. This is what makes the pattern recursive and extensible:

- A `Squad` can hold `Soldier` instances (because `Soldier` implements `IUnit`).
- A `Squad` can hold other `Squad` instances (because `Squad` implements `IUnit`).
- A `Squad` can hold any future type that implements `IUnit` without any change to `Squad`.

It is initialized inline to an empty `ArrayList<>()`, so a freshly constructed `Squad` always starts with zero members and is never `null` — members are added afterwards via `add()`.

```
    public Squad(String squadName) {  
        this.squadName = squadName;  
    }
```

The constructor takes only the squad's name. Unlike a design where members must be supplied up front, this constructor leaves `members` empty and lets the caller populate the squad incrementally via `add()`. This matches how a real formation is built: you stand up the squad, then you assign personnel to it over time.

```
    public void add(IUnit unit) {  
        members.add(unit);  
    }
```

```
}
```

Adds a member to this squad. The parameter type is the interface `IUnit`, so the caller can pass a `Soldier`, a `Squad`, or any future implementation — the method body never needs to change to support new unit types. This is the operation that actually builds the formation structure: calling `add` with another `Squad` nests one composite inside another.

```
public void remove(IUnit unit) {  
    members.remove(unit);  
}
```

Removes a member from this squad, by reference equality (the default `List.remove(Object)` behaviour, since neither `Soldier` nor `Squad` overrides `equals`). This is the structural mirror of `add()` and is standard for a Composite's container-management operations, even though the driver in this module does not currently exercise it.

```
@Override  
public void muster() {
```

Implements the `muster()` method from `IUnit`. This implementation does two things: print the squad's own name, then delegate to each member's `muster()`. The delegation is what makes the Composite pattern recursive.

```
System.out.println("Squad : " + this.squadName);
```

Prints the squad header line (e.g., `Squad : Alpha Command`). The `this.` prefix is optional here — `squadName` unambiguously refers to the instance field since there is no local variable of the same name — but it adds clarity.

```
members.forEach(IUnit::muster);
```

This single line is the heart of the Composite pattern in action:

- `members.forEach(...)` — iterates over every element currently in the member list and calls the given action on each.
- `IUnit::muster` — a method reference, equivalent to the lambda `unit -> unit.muster()`. It reads as "call `muster()` on each `IUnit` in the list."

Because each element is typed as `IUnit`, and both `Soldier` and `Squad` implement that interface, Java dispatches `muster()` polymorphically at runtime:

- If the element is a `Soldier`, `Soldier.muster()` executes — it prints the soldier's details and returns.
- If the element is a `Squad`, `Squad.muster()` executes — it prints the squad name and then recurses into *its* members' `muster()` calls.

This recursive delegation means a single top-level `muster()` call traverses the entire formation, no matter how deep it goes, with no recursion logic in the client.

```
    }  
}
```

Closes `muster()` and the `Squad` class.

BattleFormationDesignPattern.java — Driver / Client

```
package com.design.patterns.composite;
```

The driver lives in the root package, not in any of the sub-packages. This is the composition root — the one place that knows about all concrete types.

```
import com.design.patterns.composite.component.IUnit;
```

Imports the component interface. The client uses it for the `formation` variable's declared type, so that the final `muster()` call is demonstrably made through the abstraction, not through a concrete class.

```
import com.design.patterns.composite.composite.Squad;
```

Imports the concrete `Squad` class. Needed at the `new Squad(...)` construction sites and because the driver calls `Squad`-specific `add()`.

```
import com.design.patterns.composite.leaf.Soldier;
```

Imports the concrete `Soldier` class. Needed only at the `new Soldier(...)` construction sites.

```
public class BattleFormationDesignPattern {
```

The main driver class. Named after the domain and the pattern for easy identification.

```
    public static void main(String[] args) {
```

The JVM entry point. `static` because it is called without an instance. `String[] args` receives command-line arguments (unused here).

```
        System.out.println("Composite Design Pattern");
```

Prints the pattern name to standard output as a banner. This is purely cosmetic — it identifies which pattern is being demonstrated when running multiple pattern demos.

```
        Squad alphaCommand = new Squad("Alpha Command");
```

Creates the top-level composite, named `"Alpha Command"`, with an empty member list.

```
        alphaCommand.add(new Soldier("Rook", 120));
        alphaCommand.add(new Soldier("Talon", 95));
```

Creates two leaf `Soldier` instances inline and adds each directly to `alphaCommand`. This is the incremental "build the formation by calling `add`" style enabled by `Squad`'s no-members-required constructor. After these two calls, `alphaCommand` contains two soldiers.

```
        Squad reconTeam = new Squad("Recon Team");
        reconTeam.add(new Soldier("Scout-7", 60));
```

Creates a second, independent `Squad` named `"Recon Team"` and adds one leaf `Soldier` to it.

```
        alphaCommand.add(reconTeam);
```

This is the line that makes the structure recursive: `reconTeam` — itself a `Squad`, hence an `IUnit` — is added as a member of `alphaCommand`. `alphaCommand.add(IUnit)` accepts it without any special-casing, because `Squad` satisfies the same interface as `Soldier`. After this call, `alphaCommand` contains three members: two soldiers and one nested squad.

```
        IUnit formation = alphaCommand;
```

Assigns `alphaCommand` to a variable declared with the interface type `IUnit`, not the concrete `Squad` type. This is programming to the interface: the line that follows calls `muster()` purely through the abstraction, proving that the client does not need to know it is holding a `Squad` (or, transitively, a formation of squads and soldiers) to use it correctly.

```
        formation.muster();
```

The single client call that triggers the entire formation traversal. Because the runtime type of `formation` is `Squad`, `Squad.muster()` runs: it prints `"Squad : Alpha Command"`, then iterates its members — the two soldiers (printed directly) and the `reconTeam` squad (which recurses into its own `muster()`, printing `"Squad : Recon Team"` followed by its one soldier). The result is that the full formation is printed with no additional logic in the client.

```
    }
}
```

Closes `main()` and the driver class.

Why These Design Decisions

Why `List<IUnit>` and not `List<Soldier>`?

```
// Correct – holds any IUnit (Soldier, Squad, or future types)
private final List<IUnit> members = new ArrayList<>();

// Wrong – restricts members to Soldier only; cannot contain Squad
private final List<Soldier> members = new ArrayList<>();
```

If the field were typed as `List<Soldier>`, a `Squad` could never contain another `Squad`, and the recursive structure of the pattern would be impossible. The interface type is the mechanism that allows the formation to be heterogeneous.

Why `add()` / `remove()` instead of passing members through the constructor?

```
// Current code – build incrementally
Squad alphaCommand = new Squad("Alpha Command");
alphaCommand.add(new Soldier("Rook", 120));

// Alternative – build all at once
Squad alphaCommand = new Squad("Alpha Command", List.of(rook, talon));
```

An incremental `add` / `remove` API matches how a real formation is assembled: it starts empty and gains personnel over time, and personnel can later be reassigned. It also lets a `Squad` be constructed before all of its members exist yet — which is exactly what happens with `reconTeam` here: `reconTeam` is built and populated first, then attached to `alphaCommand` with `alphaCommand.add(reconTeam)`. A constructor that demanded the full member list up front could not express "build this sub-formation, then attach it to the parent" as naturally.

Why a method reference `IUnit::muster` instead of an explicit lambda?

```
// Method reference – idiomatic, concise, no noise
members.forEach(IUnit::muster);

// Equivalent lambda – correct but more verbose
members.forEach(unit -> unit.muster());
```

The method reference `IUnit::muster` is a direct pointer to the `muster()` method on the interface type. It compiles to the same behaviour as the lambda. It is preferred because it is shorter, eliminates the need to name a parameter (`unit`) that adds no information, and reads naturally as "for each element, call its `muster` method."

Why is `members` `private final` and not just package-visible?

```
private final List<IUnit> members = new ArrayList<>();
```

`private` means no other class — not even another class in the same `composite` package — can reach in and mutate the list directly, bypassing `add()` / `remove()`. `final` means the field always refers to the same `ArrayList` instance for the lifetime of the `Squad`; only the contents of that list change, never the reference itself. Together these make the encapsulation of the composite's internal structure airtight: the only way to change what a `Squad` contains is through its own `add()` / `remove()` methods.

Why does `Soldier` have no firepower validation or `Squad` no cycle check?

This module is a canonical, minimal demonstration of the Composite pattern's structural mechanics (shared interface, recursive delegation), not a production command-and-control system. Concerns such as negative-firepower validation or preventing a squad from containing itself (a cycle) are real considerations in production code but are orthogonal to the pattern itself, so they are intentionally left out to keep the example focused.

Execution Flow Trace

Here is a step-by-step trace of what happens from the moment `main()` is called to the last line of output.

```
main() called
├── System.out.println("Composite Design Pattern")
│   │   output: "Composite Design Pattern"
├── new Squad("Alpha Command")           → alphaCommand = Squad{squadName="Alpha Command", members=[]}
├── alphaCommand.add(new Soldier("Rook", 120)) → alphaCommand.members = [Soldier{Rook,120}]
├── alphaCommand.add(new Soldier("Talon", 95)) → alphaCommand.members = [Soldier{Rook,120}, Soldier{Talon,95}]
├── new Squad("Recon Team")               → reconTeam = Squad{squadName="Recon Team", members=[]}
├── reconTeam.add(new Soldier("Scout-7", 60)) → reconTeam.members = [Soldier{Scout-7,60}]
```



```

└─ alphaCommand.add(reconTeam)                → alphaCommand.members = [Soldier{Rook,120}, Soldier{Talon,95},
reconTeam]
└─ IUnit formation = alphaCommand
└─ formation.muster()                          (dispatches to Squad.muster() at runtime)
└─ System.out.println("Squad : Alpha Command")
    output: "Squad : Alpha Command"
└─ members.forEach(IUnit::muster)
    └─ [element 0 = Soldier Rook] → muster()
        output: "Soldier : Rook with firepower : 120"
    └─ [element 1 = Soldier Talon] → muster()
        output: "Soldier : Talon with firepower : 95"
    └─ [element 2 = reconTeam] → Squad.muster() called recursively
        └─ System.out.println("Squad : Recon Team")
            output: "Squad : Recon Team"
        └─ members.forEach(IUnit::muster)
            └─ [element 0 = Soldier Scout-7] → muster()
                output: "Soldier : Scout-7 with firepower : 60"

```

The key moment in this trace is the polymorphic dispatch inside `forEach`. The list sees each element as `IUnit`. When `muster()` is called on an element, the JVM looks up the actual runtime type of that object and calls its implementation. For Rook, Talon, and Scout-7, the runtime type is `Soldier`, so `Soldier.muster()` runs and terminates. For `reconTeam`, the runtime type is `Squad`, so `Squad.muster()` runs — which in turn calls `forEach` on `reconTeam`'s own members, recursing one level deeper. If `reconTeam` itself contained another `Squad`, the same mechanism would recurse again, to any depth.

Expected Output

Running the driver produces exactly this console output (captured from an actual build and run of this module):

```

Composite Design Pattern
Squad : Alpha Command
Soldier : Rook with firepower : 120
Soldier : Talon with firepower : 95
Squad : Recon Team
Soldier : Scout-7 with firepower : 60

```

How to Run

From inside this module directory:

```

JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64 mvn -o clean compile
java -cp target/classes com.design.patterns.composite.BattleFormationDesignPattern

```

(Drop `-o` if offline dependency resolution fails; the parent POM's dependencies should already be cached from a prior build.)

Real-World Use Cases

1. Military Command and Control Structures

The domain modeled here. A battlefield force is a Composite: individual soldiers are leaves (they take orders, they have no subordinate units), squads and platoons are composites (they contain soldiers and other squads). Operations like "muster the formation," "report total headcount," or "broadcast an order" all traverse the hierarchy uniformly — they do not special-case an individual soldier vs. a squad at the call site. The traversal logic is in the recursive descent, not in the command implementation.

2. Sports Team Roster with Sub-Units

A sports league roster is a tree. At the leaves are individual athletes. At the branches are line units or squads (an offensive line, a special-teams unit) that group athletes, and those groupings can themselves be grouped into a full

roster. An operation like `getTotalSalaryCap()` or `printLineup()` naturally recurses: a leaf athlete returns their own value, a unit returns the sum over its members.

3. Bill of Materials (BOM) in Manufacturing

A manufactured product is described by a Bill of Materials — a hierarchical list of components. A finished good (e.g., a laptop) is a composite containing sub-assemblies (motherboard, display panel, chassis) which are themselves composites containing individual parts (capacitors, screws, hinges) which are leaves. Operations like `getTotalCost()`, `getTotalWeight()`, and `getPartCount()` are all Composite-pattern recursive computations: each composite sums the result across its children, each leaf returns its own value.

4. Orchestral Score with Sections

An orchestral score groups individual instrument parts (leaves) into sections (strings, brass, woodwinds), and a full score is a composite of sections. Rehearsal tooling that needs to "mute this group" or "transpose this group" operates uniformly on a single instrument part or an entire section through the same interface, without the tool caring which one it was handed.

5. Tournament Bracket Structure

A tournament bracket is a tree of matches: a leaf is a single scheduled match between two competitors, and a composite round groups several matches (or sub-rounds) together. An operation like `printSchedule()` or `countRemainingMatches()` recurses over the bracket uniformly, regardless of how many rounds deep the tournament goes.

6. Supply Convoy Formation

A logistics convoy is composed of individual vehicles (leaves) and sub-convoy (composites) escorting a larger movement. A `reportFuelConsumption()` operation sums fuel use across an entire convoy tree the same way whether it is applied to a single truck or the whole formation, because every node — vehicle or sub-convoy — implements the same reporting contract.

Extending This Example — Deeper Nesting

The current demo already has two levels of nesting: `alphaCommand` directly contains two soldiers and a `reconTeam` sub-squad, and `reconTeam` contains one soldier. The pattern extends to any depth with no changes to `IUnit`, `Soldier`, or `Squad` — only the driver needs to grow.

```
public class BattleFormationDesignPattern {
    public static void main(String[] args) {
        System.out.println("Composite Design Pattern – Deeper Nesting");

        Squad alphaCommand = new Squad("Alpha Command");
        alphaCommand.add(new Soldier("Rook", 120));
        alphaCommand.add(new Soldier("Talon", 95));

        Squad reconTeam = new Squad("Recon Team");
        reconTeam.add(new Soldier("Scout-7", 60));
        alphaCommand.add(reconTeam);

        Squad forwardObservers = new Squad("Forward Observers");
        forwardObservers.add(new Soldier("Marker-2", 40));
        reconTeam.add(forwardObservers);

        IUnit formation = alphaCommand;
        formation.muster();
    }
}
```

Expected output:

```
Composite Design Pattern – Deeper Nesting
Squad : Alpha Command
Soldier : Rook with firepower : 120
Soldier : Talon with firepower : 95
Squad : Recon Team
Soldier : Scout-7 with firepower : 60
Squad : Forward Observers
Soldier : Marker-2 with firepower : 40
```

Trace of `formation.muster()` for this extended tree:

```

alphaCommand.muster()
└─ prints "Squad : Alpha Command"
   └─ forEach over [Rook, Talon, reconTeam]
      └─ Rook.muster() → "Soldier : Rook with firepower : 120"
         └─ Talon.muster() → "Soldier : Talon with firepower : 95"
            └─ reconTeam.muster() (Squad.muster() called recursively)
               └─ prints "Squad : Recon Team"
                  └─ forEach over [Scout-7, forwardObservers]
                     └─ Scout-7.muster() → "Soldier : Scout-7 with firepower : 60"
                        └─ forwardObservers.muster() (Squad.muster() called recursively again)
                           └─ prints "Squad : Forward Observers"
                              └─ forEach over [Marker-2]
                                 └─ Marker-2.muster() → "Soldier : Marker-2 with firepower : 40"

```

Notice that the driver code has zero `instanceof` checks and zero conditional logic. It simply builds the formation with `add()` calls and triggers one `muster()` at the top. Adding a third level of nesting requires only a few more `add()` calls in the driver — not a single change to `Squad` or `Soldier`.

Summary

The Composite Design Pattern solves the part-whole hierarchy problem by making composite containers and leaf nodes indistinguishable through a shared interface. The four classes in this implementation map directly to the pattern's three roles:

Role	Class	Responsibility
Component	<code>IUnit</code>	Declares the common <code>muster()</code> contract
Leaf	<code>Soldier</code>	Terminal unit; fulfills the contract by printing its own data
Composite	<code>Squad</code>	Branch unit; fulfills the contract by delegating to all members, and exposes <code>add()</code> / <code>remove()</code> to manage them
Client	<code>BattleFormationDesignPattern</code>	Builds the formation with <code>add()</code> calls and calls <code>muster()</code> once, through the <code>IUnit</code> abstraction, without caring about unit types

The pattern's power is proportional to the depth and variety of the formation. In this small three-node demo the benefit is modest. In a real command structure with thousands of personnel across many nested squads and platoons — or a manufacturing BOM with thousands of parts, or a tournament bracket with many rounds — the elimination of `instanceof`-based branching from every tree traversal operation becomes significant. The Composite pattern pushes type-awareness down into the objects themselves and out of the code that uses them.